

MIT AI2 204

IoT with MIT App Inventor

Fundamental

X. Tang

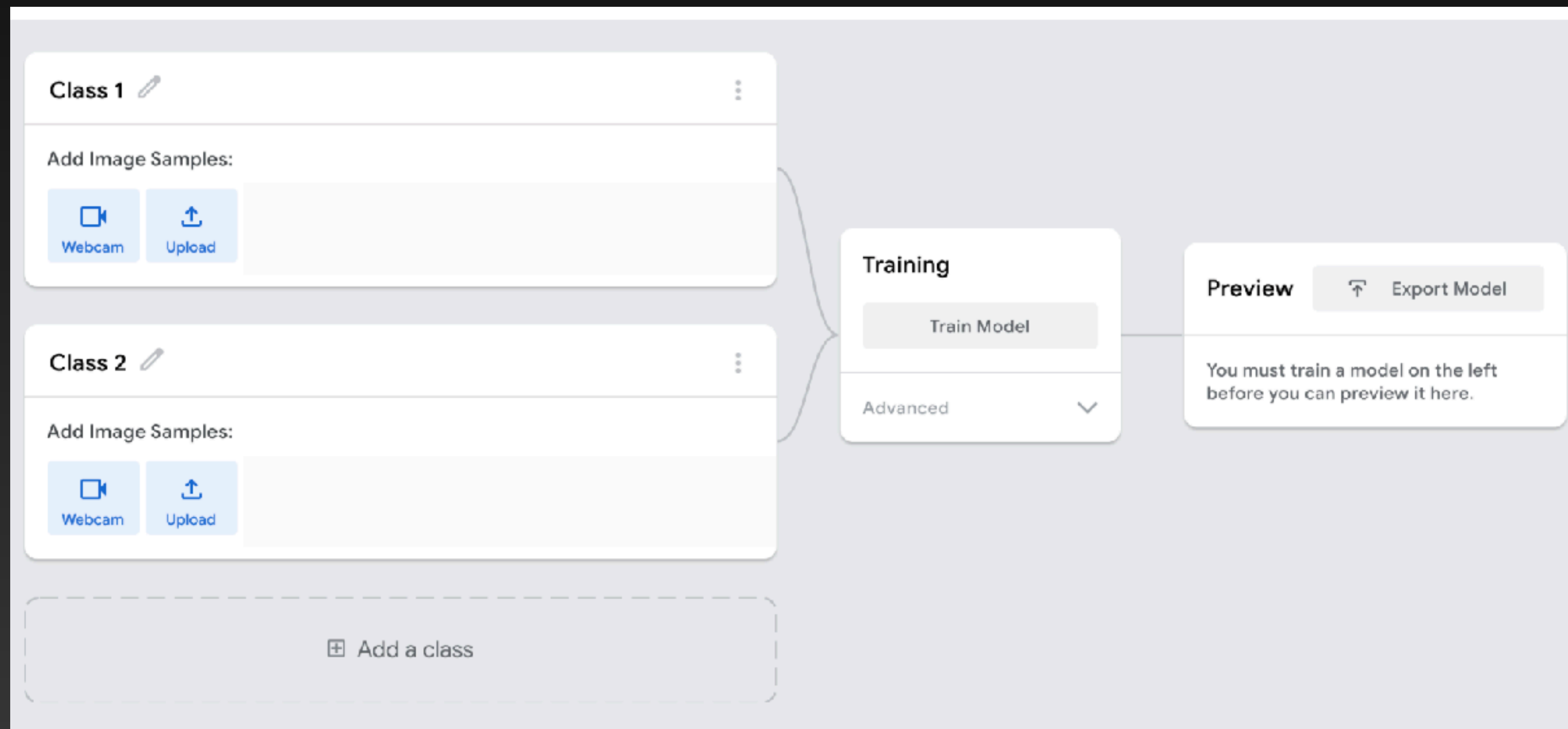
Get started with teachable machine

This project has two parts to it: first you build a model on Teachable Machine, and then you use that model in a game you'll build on your computer.

Teachable Machine is a tool made by Google that creates machine learning models in your browser. You'll create a model to recognise images, but it can also create models that recognize sounds, or body poses.

Get started with teachable machine

<https://teachablemachine.withgoogle.com/train/image>



Get started with teachable machine

This is an untrained model, with two classes. Of course, since you need to recognize 'rock', 'paper', and 'scissors', you're going to need three classes.

Click the **Add a class** button to add a third class to your model.



 Add a class

Get started with teachable machine

Next, give your classes proper labels that match what they're going to be trained to recognize.

Use the pencil beside each class name to rename them as follows:



- Change 'Class 1' to 'rock'
- Change 'Class 2' to 'paper'
- Change 'Class 3' to 'scissors'

It's important to match this order, because some of the code provided to help your game run assumes that the classes are ordered like this.

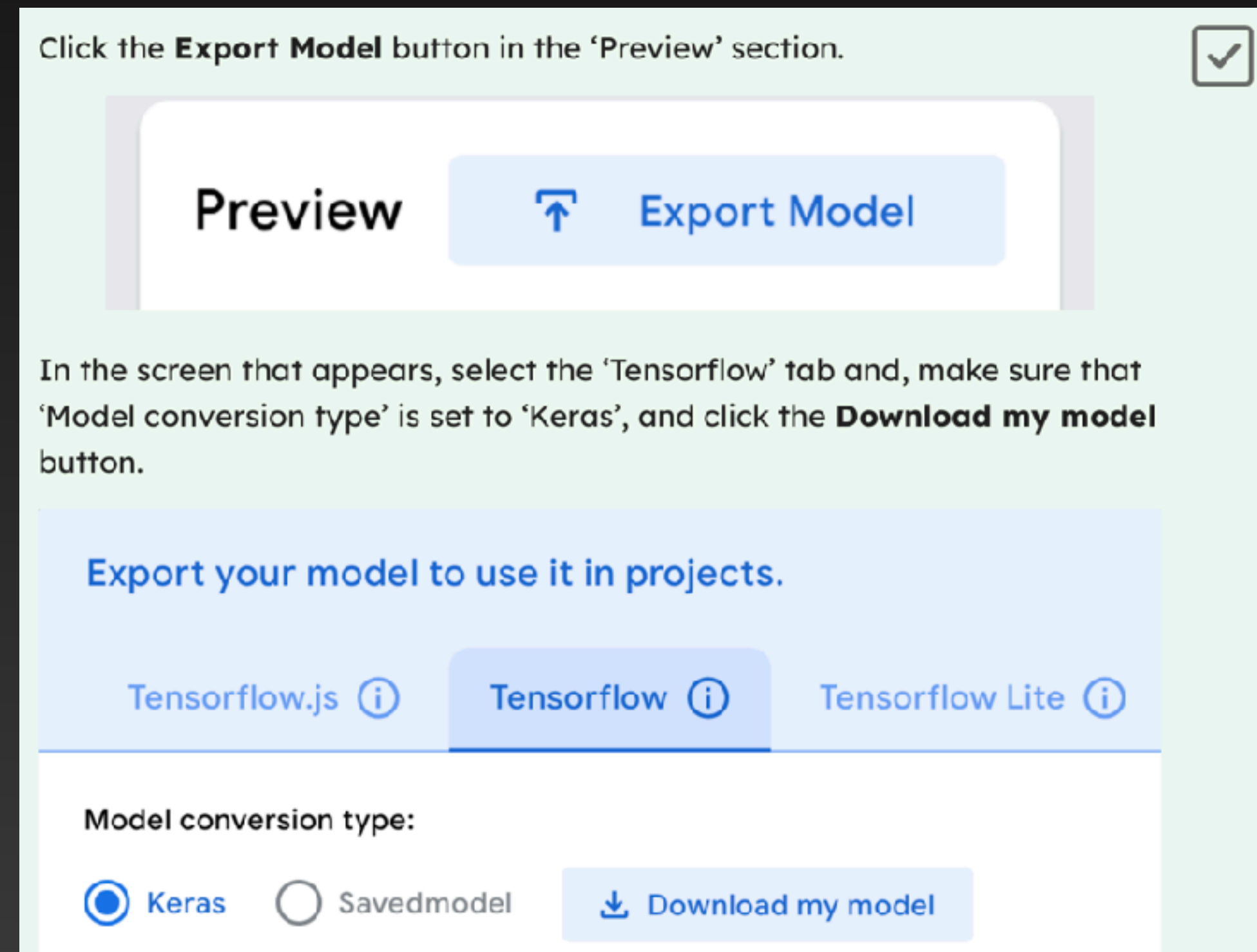
Get started with teachable machine

Create your model

The training will take a short time, during which you can watch Teachable Machine count the epochs it's running through. You need to leave your browser open on the Teachable Machine tab, or the model will stop training. Once the training is complete, you'll see a preview of the model where you can test it by making gestures at your camera and seeing how likely your model thinks that gesture is to be each of 'rock', 'paper', and 'scissors'. The highest percentage indicates the 'guess' that your game would make if it saw that gesture.

Get started with teachable machine

Once your model works well enough — it will never be right all the time, but being right most of the time is pretty good — it's time to export the model so you can use it as part of your game.



Build your game

Your completed model now needs to be inserted into a desktop Python application to make a working game. A simple version of rock, paper, scissors, where the computer will use the player's guess to cheat, has been written for you to use. You are focusing on the machine learning aspects of the project.

Build your game

For windows:

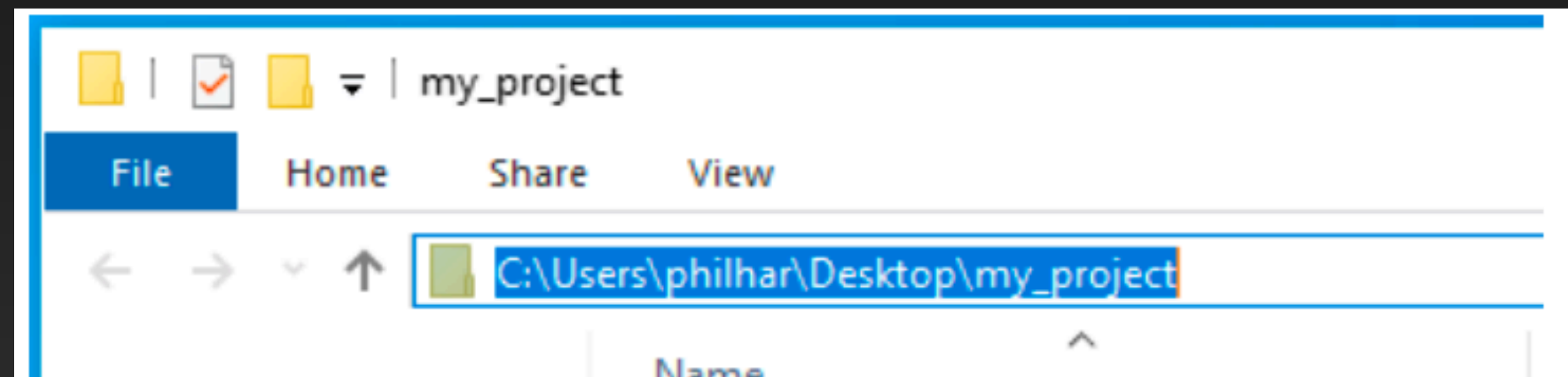
Download the [starter zip file](#) and unzip it somewhere you'll remember on your computer. If you can't think of a location, just put it on your desktop. This isn't the best place to keep things in the long term, but it's fine when you're working on them.

Next, you need to install the libraries you will use in this project. For this, you'll need to use the command line interface (CLI) — a program that controls your computer by typing text commands into a window. The command line interface is called 'command prompt' in Windows.

Build your game

For windows:

The easiest way to find the path to a directory you know in Windows is to open the folder in Windows Explorer, as you would normally, and click into the navigation bar at the top of the window. The full path for the folder should become visible and you can then copy it.



The screenshot shows a Windows Explorer window titled 'my_project'. The address bar displays the full path 'C:\Users\philhar\Desktop\my_project'. Below the address bar, there is a list of files and folders. The path is highlighted in blue.

In the CLI, navigate to the directory you just unzipped by entering the following command, and replace `[directory_path]` with the path to your directory. ☒

```
cd [directory_path]
```

Build your game

For windows:

Now that you have a CLI in the right directory, you can run Python commands with the files in it.

The command to install the libraries you need uses pip, a tool to fetch Python code written by other people from the internet and set it up so you can use it in your projects. It's important to use pip to install libraries rather than just downloading them: some libraries need other libraries to work (these libraries are called their dependencies) and pip will automatically install those too.

Conveniently, pip can be given a list of all the libraries a project needs, and told to install them all at once. These are usually included in a file called `requirements.txt`, as they have been with the starter code provided here.

```
pip install requirements.txt
```

Build your game

For Mac OS:

The script may take several minutes, or more, to complete the setup depending on the speed of your computer and your internet connection. Once it has finished, you will have a new directory inside your home directory, called `amazing_image_identifier`. This is the directory you'll work in.

Open the CLI on your computer and type the command below in:



```
cd ~
```

Now press the Return key.

To change directory to the `amazing_image_identifier` directory, type the following command into your CLI and press the Return key.



```
cd amazing_image_identifier
```

To download and run the program, type (or copy and paste) the command below into your CLI and press the Return key.



```
curl -L https://rpf.io/proj-rps | sudo bash -s $USER
```

Build your game

There's a lot of code already provided for you, but it's all to create the game and capture an image from your computer's camera. First, run the program as it is to get an idea of how the game will work.

Go back to your CLI window and type the following command to run the program. Remember it, as you'll need to run the program several times.



If you're using Windows or Linux

```
python project.py
```

Then press the Return key to run the program.

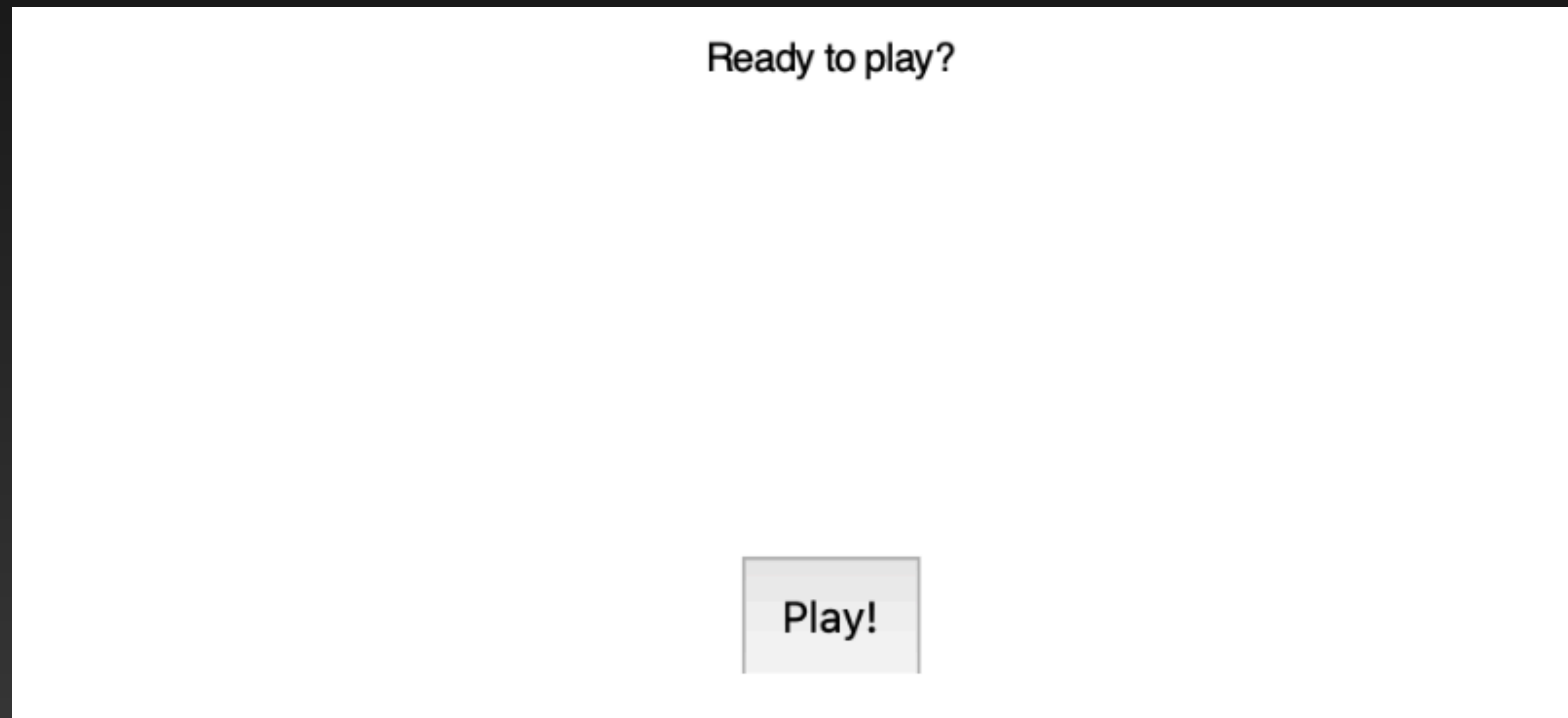
If you're using macOS

```
python3 project.py
```

Then press the Return key to run the program.

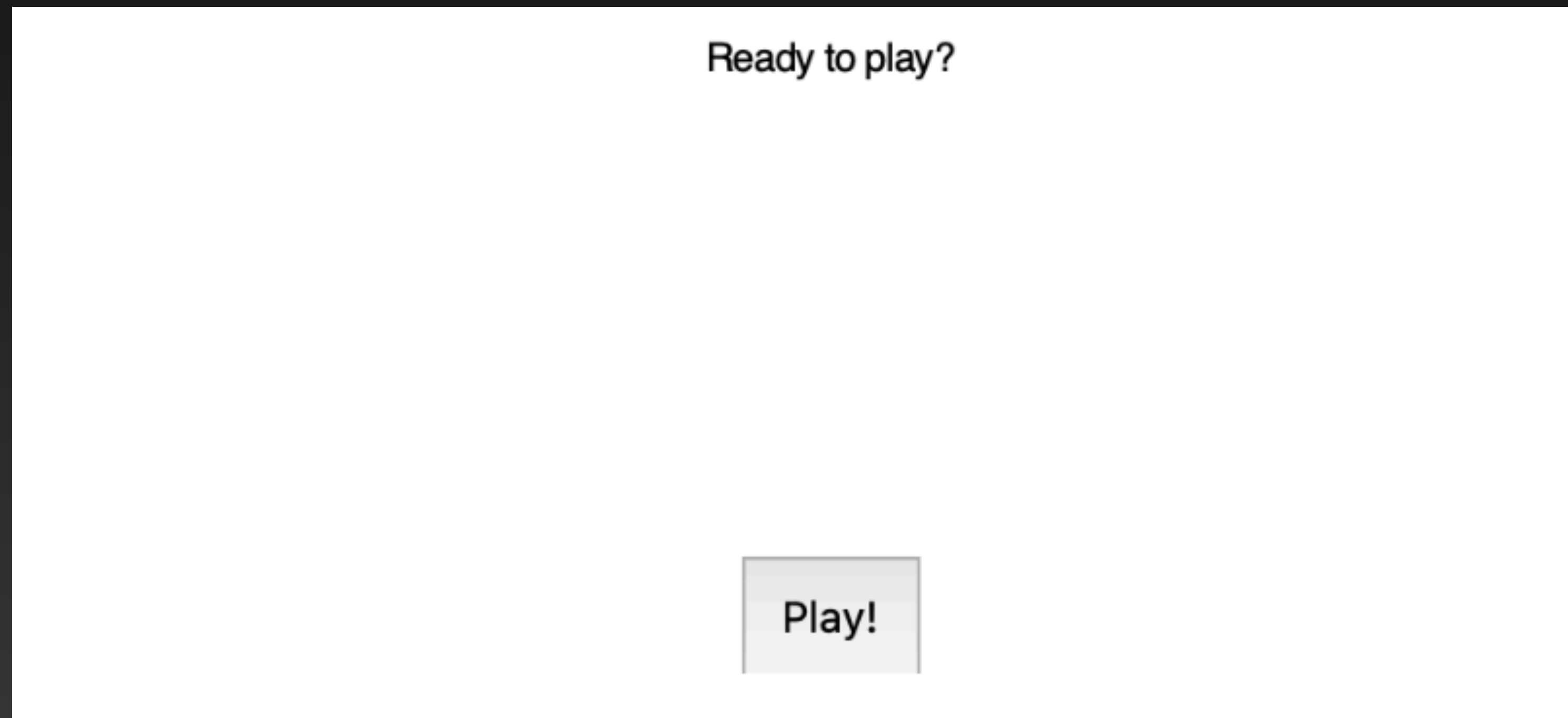
Build your game

It may take a moment, but when your program runs you should see something like the image below.



Build your game

It may take a moment, but when your program runs you should see something like the image below.



Connect your model to the game

First, find the model you downloaded from Teachable Machine. It should be in a file called `converted_keras.zip`. It may take a moment, but when your program runs you should see something like the image below.

Unzip `converted_keras.zip` and open the resulting folder. Copy the `keras_model.h5` file to the project directory that contains the code for the game.

Note that the `labels.txt` file in the unzipped `converted_keras` directory contains the list of human-readable labels and the node numbers they match to on the model's output layer.

Connect your model to the game

For a larger number of categories, you would have Python read this file and automatically load them but, as there are only three categories in this model, the labels have already been included in the game code for you, in the list called **THROWS**.

Find the function `get_player_throw`. On a line above it, add the following:



```
model = tf.keras.models.load_model('keras_model.h5', compile=False)
```

Connect your model to the game

This will load your model into the game. Now you need to add a function that uses the model to get players' throws — their choice of 'rock', 'paper', or 'scissors'.

Update the `get_player_throw` function with the following code which will load the image the game has captured, then pass it to the model for a prediction. That prediction is then returned, so it can be used by the game code, instead of the default 'rock' that the game came with!

Connect your model to the game

```
def get_player_throw():  
    image = tf.keras.preprocessing.image.load_img(IMG_NAME, target_size=(IMAGE_SIZE,  
IMAGE_SIZE))  
    image = tf.keras.preprocessing.image.img_to_array(image)  
    image = np.expand_dims(image, axis=0)  
  
    prediction_result = model.predict(image)  
  
    best_prediction = THROWS[np.argmax(prediction_result[0])]  
  
    return best_prediction
```

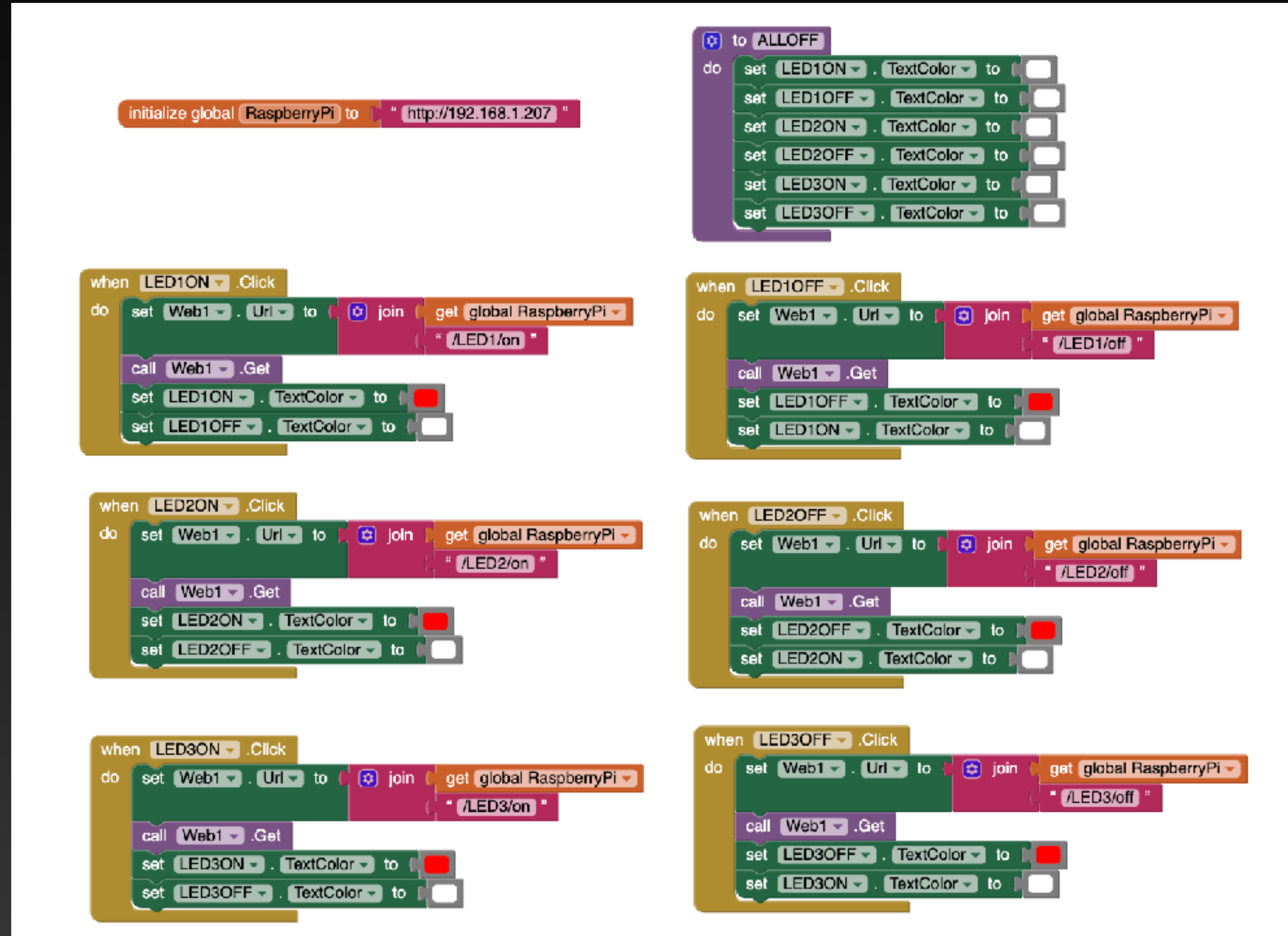
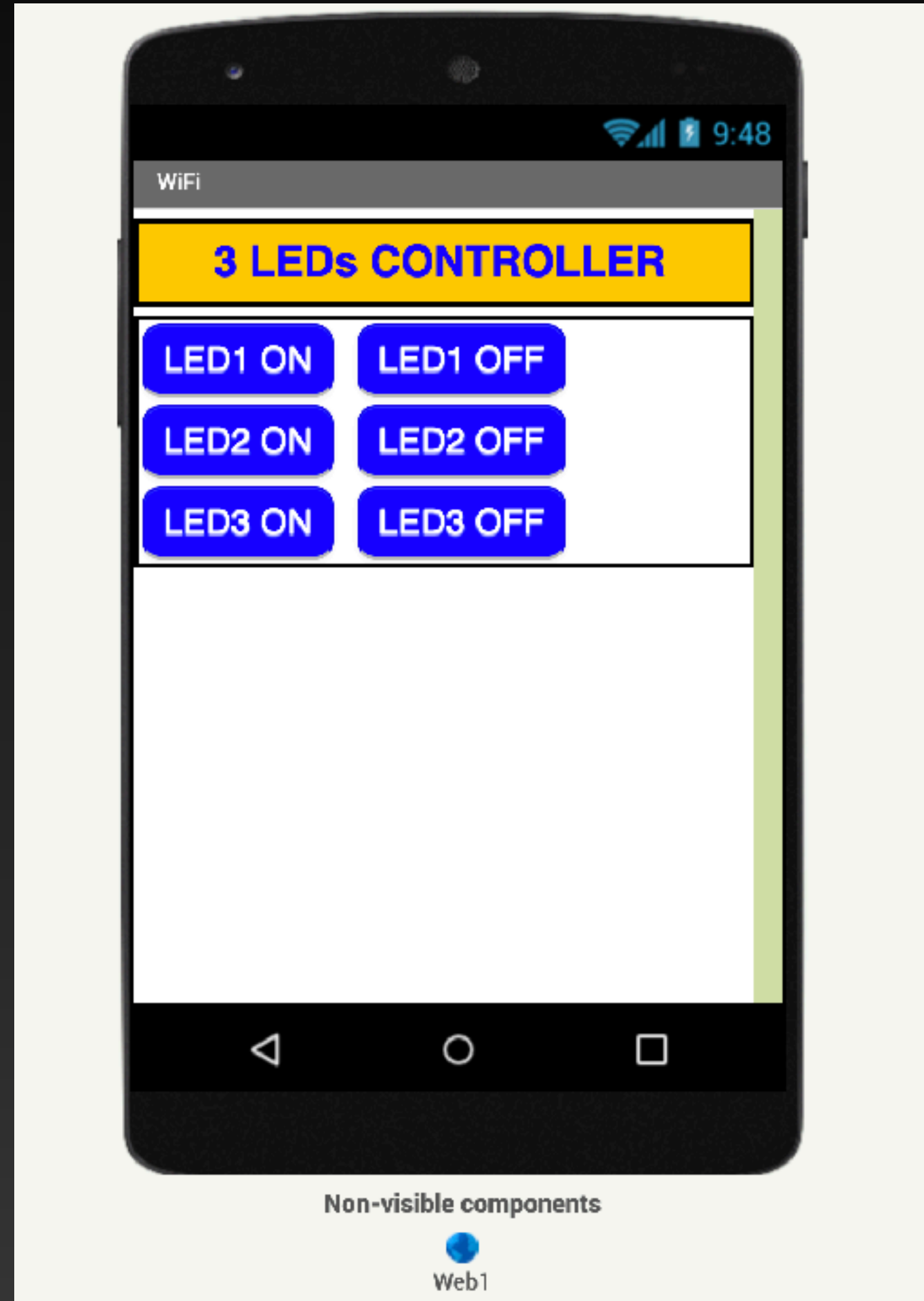
Connect your model to the game

Run the program and play with it a few times! If you're not happy with the quality of its guesses, think about any differences between the training data and the inputs you give your game — maybe the light has changed, or you're wearing different clothes, etc. If you need to, you can record more training data in Teachable Machine, train your model again, and then download it and replace the model in the game with an updated version.

IoT with MIT App Inventor

Additional Resources

Control Multiple LEDs through Wifi



Control Multiple LED through Bluetooth

Button Clicked	Command sent to Rpi	GPIO port
LED1 ON	11	GPIO 2
LED1 OFF	10	
LED2 ON	21	GPIO 3
LED2 OFF	20	
LED3 ON	30	GPIO 4
LED3 OFF	31	

Control Multiple LED through Bluetooth

```
import socket    # import socket library
import RPi.GPIO as GPIO
GPIO.setwarnings(False) # disable warnings
GPIO.setmode(GPIO.BCM) # set BCM pin numbering
```

```
LED1 = 2          # LED on GPIO 2
LED2 = 3          # LED on GPIO 3
LED3 = 4          # LED on GPIO 4
```

```
GPIO.setup(LED1, GPIO.OUT) # conf LED1 as output
GPIO.setup(LED2, GPIO.OUT) # conf LED2 as output
GPIO.setup(LED3, GPIO.OUT) # conf LED3 as output
```

```
GPIO.output(LED1, 0) # turn off LED1 to start with
GPIO.output(LED2, 0) # turn off LED2 to start with
GPIO.output(LED3, 0) # turn off LED3 to start with
```

```
Port = 1
MAC = 'DC:A6:32:8E:0F:3E'

s =
socket.socket(socket.AF_BLUETOOTH,socket.SOCK_STREAM,socket.BTPROTO_RFCOMM)
s.bind((MAC, Port))
s.listen(1)
client, addr = s.accept()
```

Control Multiple LED through Bluetooth

```
#
# turn on the LED if command is : 1
# turn on the LED if command is : 0
#

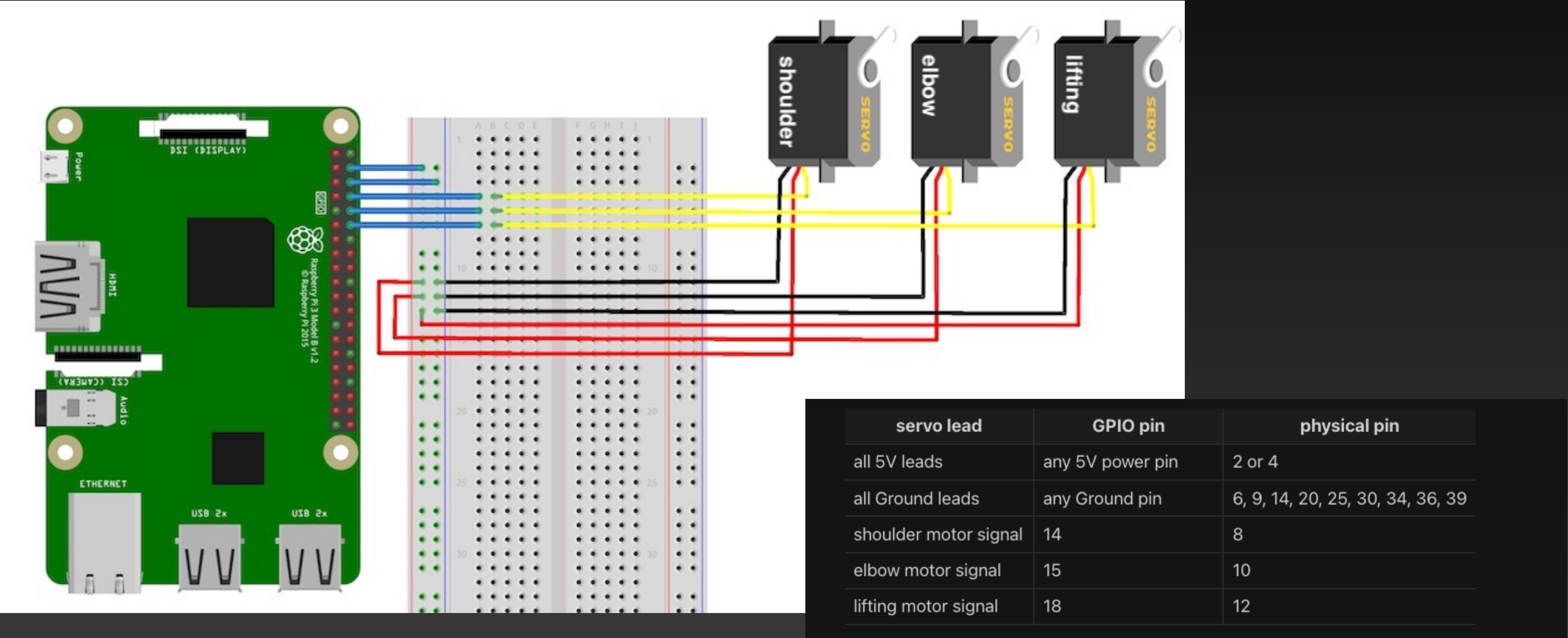
try:
    while True:
        data = client.recv(1024)          # Recieve data bytes
        if data.decode('utf-8') == '11':  # 11 received
            GPIO.output(LED1, 1)          # LED ON
        elif data.decode('utf-8') == '10': # 10 received
            GPIO.output(LED1, 0)          # LED OFF
        if data.decode('utf-8') == '21':  # 21 received
            GPIO.output(LED2, 1)          # LED ON
        elif data.decode('utf-8') == '20': # 20 received
            GPIO.output(LED2, 0)          # LED OFF

        if data.decode('utf-8') == '31':  # 30 received
            GPIO.output(LED3, 1)          # LED ON
        elif data.decode('utf-8') == '30': # 31 received
            GPIO.output(LED3, 0)          # LED OFF

    except KeyboardInterrupt:             # Keyboard interrupt
        client.close()
        s.close()
```

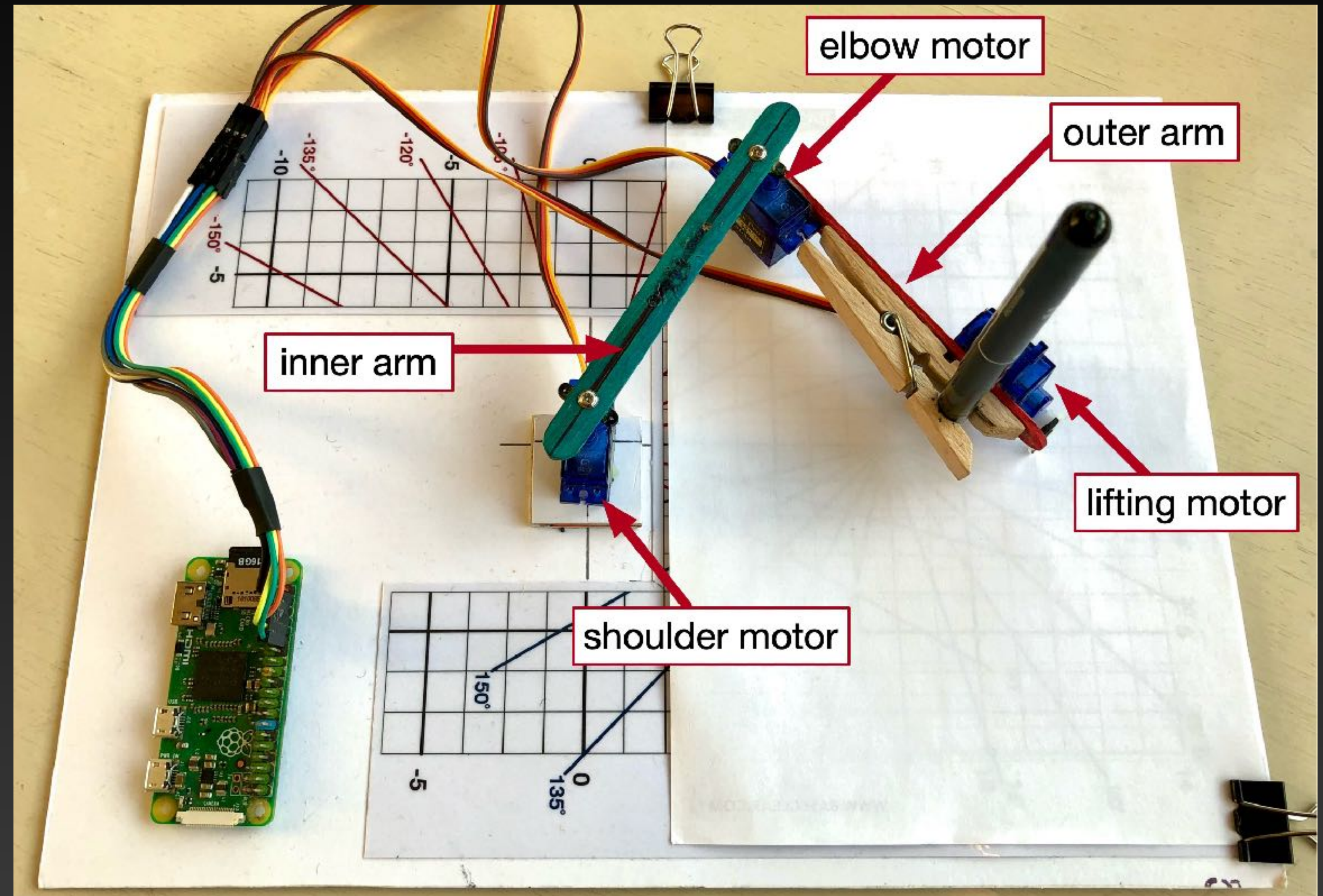
Capstone project - plotter machine

Wiring carefully, double check each connection all the way from the servo to the Raspberry Pi.



Capstone project - plotter machine

Finish building your project.



Homework

IMPORTANT!

Carefully go through the plotter machine construct link and finish assembling before tomorrow's class. We will focus on the software part of the project in class.

<https://www.brachiograph.art/en/latest/tutorial/construct/>